Summary of Pre-Training 1/27/26

Rishith Seelam (rseelam), Joshua Hsueh (jhsueh), Jimmy Dai (jimmydai), Olaf Dsouza (olafpd)

WLB-LLM: Workload-Balanced 4D Parallelism for Large Language Model Training

Problem and Motivation

Currently, the largest LLM training tasks involve thousands of GPUs operating for multiple consecutive months. A high degree of parallelism is necessary to create reasonable training times and because multi-billion parameter models cannot fit in the memory of a single hardware device. To effectively tackle problems of this magnitude, 4D parallelism is used: Data Parallelism (DP), Pipeline Parallelism (PP), Tensor Parallelism (TP), and Context Parallelism (CP). However, partitioning the input for parallel computation is a non-trivial problem that significantly affects training latency.

DP, TP, and CP each rely on synchronization operations between cooperating hardware devices while training. Thus the latency is proportional to the slowest device. With PP, the latency is proportional to the most computationally intensive microbatch present in the pipeline. In both cases, imbalanced problem partitioning leads to idle hardware resources.

The status quo in training is to partition an input into equal token-length subproblems. This involves appending multiple documents together and padding as necessary. However, this fails to consider heterogeneous computational complexity per token. Specifically, the attention computation is quadratic in document length, so larger documents have an elevated per-token computational cost. Furthermore, a token at the start of a document has a lower computational cost than one at the end, which is notable for CP. This document packing scheme leads to significant workload imbalances.

WLB-LLM addresses the issue of imbalanced workloads in the LLM training architectures. It proposes a novel workload-aware variable-length document packing method and a fine-grained per-document sharding strategy at the CP level.

Related Works

Distributed LLM Training Frameworks: To address training massive models, context parallelism was introduced. Existing frameworks were struggling to scale with growing context windows. Splitting long sequences into chunks alleviates memory pressure caused by this growth. However, per-token arithmetic intensity was overlooked in early implementations of CP. This is a key issue that WLB-LLM tackles.

Input Padding and Packing for LLM Training: Model stages expect fixed-size tensors, which input data may not match. Thus, padding or packing is necessary. Padding appends zeros to meet sizing. While simple, it wastes computation. Packing combines shorter documents, minimizing the amount of padding required. A new masking step is needed to ensure tokens only attend to tokens in the same document. Due to improved efficiency, packing has become the standard approach, and WLB-LLM advances the status-quo in packing strategies.

Solution Overview

WLB-LLM proposes a workload-aware variable-length document packing mechanism. The per-document computation cost is quadratic in document length. This is because the attention cost is quadratic. A basic packing scheme may concatenate documents such that the combined token length matches the context window. However, large documents will have significantly longer computation time, causing workload imbalance. Thus, WLB-LLM departs from fixed-length problem sizes in favor of variable-length workload packing.

WLB-LLM formulates the packing problem into an Integer Linear Program that minimizes the estimated computation cost. This is NP-Hard, and solving it at runtime is an unacceptable cost. Instead, a heuristic approach is used to efficiently approximate the packing scheme.

Rearranging documents to follow a specific pattern reduces the randomness of inputs, which may impact model loss and convergence. Thus, WLB-LLM generates near-optimal packing while minimizing document shifting. Notably, large documents are uncommon but account for most of the workload imbalance. Thus, WLB-LLM uses an outlier delay system where large documents are placed in queues of buckets, with each bucket corresponding to a range of lengths. When sufficient documents are accumulated in a queue, the queue is flushed such that the evenly-sized documents are spread across the training architecture. This, along with variable length packing, allows for workload balancing with minimal data rearrangement.

WLB-LLM also proposes a fine-grained per-document sharding strategy at the CP level. Micro-batches are sharded across CP workers. The status-quo is a Per-Sequence Sharding strategy that splits the entire sequence in 2xCP_size chunks. However, a multi-document micro-batch along with unlucky document and shard boundaries could lead to a significant load imbalance, given that each token attends to all preceding tokens in a document. Thus, WLB-LLM proposes Per-Document Sharding, which splits each document into 2xCP_size chunks, giving each worker a symmetric pair. This eliminates any potential imbalances, but comes with additional costs. Smaller chunk sizes lead to wasted computation at the tile level given fixed tile sizes. Additionally, there is more memory transfer overhead with smaller problem sizes. Thus, while balancing workload, Per-Document Sharding is not always faster. WLB-LLM adopts an adaptive sharding strategy that estimates costs via offline profiling to select an optimal strategy at runtime.

Combining all strategies, WLB-LLM achieves tangible speedups relative to the baselines. While speedups are dependent on factors like context window size, speedup values were found in the range of 1.19x-1.40x. This is accomplished while preserving model fidelity, having an only 1.6% average increase in training loss.

Limitations

The 1.6% increase in training loss, while low, poses some concerns. Beyond the explanation of reduced randomness in input, it is unclear why delaying certain large documents leads to different performance. Additional analysis regarding the relationship between packing schemes and model performance would be valuable. Specifically, the change in loss should be documented as a function of various architecture parameters (i.e. context window size).

It’s noted in the paper that the performance optimizations from this approach seem to scale with the size of the context window: 1.4x for 160k, while 32k sees 1.03x. It’s worth considering the impact of this approach on smaller context windows that may introduce tile-level

waste that could potentially slow down performance if the sharding pushes the query length(Q_len) to below the tile size.

Future Research Directions

The current WLB-LLM adaptive context parallelism scheme chooses between Per-Document or Per-Sequence sharding. A hybrid approach, where large and short documents are handled differently could achieve better results. This hybrid approach is a potential avenue of future research.

Zero Bubble (Almost) Pipeline Parallelism

Problem and Motivation

As models grow larger, it becomes much harder for single machines to support the memory requirements during training. Pipeline parallelism (PP) is one key technique used to increase the efficiency of large-scale distributed training for deep neural networks, leveraging a horizontal scaling approach where the training is partitioned across multiple devices.

Traditional neural network training consists of a sequential forward and a backward pass, which causes synchronous pipeline schedules to suffer from unavoidable idle time, deemed as “pipeline bubbles” by the paper. This problem is important as suboptimal scheduling results in poor resource utilization, which almost always results in poor scalability. As model sizes and data centers continue to grow, even small inefficiencies and downtimes can compound into significant resource waste. Eliminating or substantially reducing pipeline bubbles without breaking synchronous training semantics would be a great advancement not only regarding the efficiency but also the scalability of distributed DNN training.

Related Works

There have been a plethora of works attempting to mitigate pipeline bubbles. A few notable ones include:

GPipe: Although Zero bubble works by splitting backward computation into distinct pieces, it is far from the only approach. Another approach discussed in the GPipe paper works by splitting batches into many microbatches, which gives it the flexibility to pipeline and improve utilization. A downside of this approach is that it has to recompute some backward pass activations which adds additional compute cost but alleviates some memory requirements.

Asynchronous pipeline parallelism (PipeDream, PipeMare): The Pipedream paper describes an approach to improve GPU utilization that is centered around introducing asynchrony to pipeline stages that lets them process different microbatches at different times. This is made safe by the use of weight versioning and is able to improve utilization at the cost of increasing the complexity of the system and breaking the synchronous training guarantee. Another approach is described in the Pipemare paper which is similar in that it allows asynchronous computation but tolerates computations based on stale weights. This premise is outright rejected by the ZeroBubble paper, which preserves correctness requirements.

1F1B: 1F1B is a simpler approach to dealing with bubbles which works by computing stages in this order: forward(i) -> backward(i-k) ->...(after a warm up phase that fills the pipeline. This approach however, can’t eliminate the bubbles and only reduces their size. ZeroBubble

builds on 1F1B by eliminating its primary drawback of treating backward as an atomic operation which provides it much smarter scheduling.

Solution Overview

None of the previously existing synchronous PP methods eliminate bubbles entirely. The remaining idle time is widely considered unavoidable due to the granularity at which computation is scheduled - namely, treating the backward pass as a single indivisible operation. This paper challenges that assumption by splitting the backward pass into two distinct operations: the gradient w.r.t to the inputs (denoted B) and the gradient w.r.t to the weights/parameters (denoted W). Grouping these together in a singular operation can introduce unnecessary dependencies in the pipeline as devices must wait for both backward computations in a stage to complete before proceeding onto the next stage.

These two components introduce additional scheduling flexibility within the pipeline. As the W computation be delayed or reordered after the corresponding B computation. This allows W computations to be strategically scheduled into pipeline bubbles, effectively filling idle gaps without violating dependency constraints. Using this intuition, the authors designed two handcrafted pipeline schedules to conceptually demonstrate the significant bubble reductions, even showing the possibility of a zero-bubble schedule when memory constraints were completely relaxed, both under ideal assumptions that the time cost for each stage are identical. In practice, this assumption almost never upholds and things like uneven execution times and inter-device communication costs can all cause significant latency in the pipeline stream.

To address these challenges, the authors propose an automatic pipeline scheduling algorithm that searches for an optimal or near-optimal schedule by dynamically determining how many forward passes to perform during warm-up and strategically placing backward parameter (W) computations to fill idle gaps. This algorithm leverages a heuristic strategy given by the number of pipeline stages, the number of microbatches, memory limitations, and running time estimations for each stage and communication latencies. The authors also note that the problem can be systematically formulated and solved as Integer Linear Programming, allowing for further small-scale refinement after the heuristic initialization.

A further obstacle to achieving zero bubbles arises from synchronization during the optimizer step. The authors observed that many of the checks made in the optimizer step often are not triggered due to the existing robustness of the training algorithm. To address this, the paper introduces a post-validation optimizer strategy where each stage locally applies its optimizer update with a deferred consistency check and later rolls it back if an inconsistency is later detected, preserving synchronous semantics while eliminating synchronization induced pipeline stalls.

Limitations

One key limitation of the proposed approach is its increased activation memory requirement when attempting to achieve near-zero pipeline bubbles. The authors show that eliminating bubbles almost entirely typically requires approximately twice the activation memory of the standard 1F1B schedule. While they argue that this trade-off is reasonable for large-scale training systems, especially when combined with memory-saving techniques such as ZeRO or

tensor parallelism, it nonetheless constrains the applicability of the algorithm on memory-limited hardware.

The effectiveness of the automatic pipeline scheduling algorithm also depends on accurate profiling of computation and communication costs. The paper does not deeply explore how sensitive the scheduler is to profiling errors, runtime variability, or changes in system behavior during long training runs, which leaves open questions about robustness in highly dynamic environments.

Future Research Directions

Exploring the effectiveness of the proposed scheduling techniques on other model architectures, such as multimodal networks, can help establish the generality of the scheduling algorithm as the authors primarily focused on transformer-based LLMs. Additionally, more formal analysis of the optimizer post-validation mechanism could strengthen theoretical understanding of its correctness and robustness, further solidifying the case for synchronous zero-bubble pipeline parallelism.

Summary of Class Discussion

- (WLB) Does order of documents affect model performance? Does it affect generalizability?
    - The change in loss was negligible with the novel packing scheme. However, it might be good to include additional benchmarks. In general, deep learning lacks interpretability, so it is hard to specifically determine why repacking changed loss.
- (WLB) Does the architecture still hold well if the distribution of document lengths was different?
    - Systems often need to make assumptions based on their data and trends. A new data distribution may require retuning heuristics or an entirely new architecture.
- (Both) What is the tradeoff between using a heuristic or solving the ILP?
    - If the heuristic does reasonably well, then there is no need for an expensive, complete solution. ILP solving can be expensive to do at runtime.
- (WLB) Will the imbalance in PP or CP dominate as the context window grows?
    - CP imbalance increases the most as context window grows.
- (Both) Many of these parallelism optimizations are reliant on the hardware infrastructure of certain machines, can these results be generalized?
    - Depending on hardware, some of the optimizations may actually be suboptimal due to communication bandwidth limitations.
- (ZeroBubble) The paper proposes a kind of “single shot” heuristic where all heuristic strategies are performed beforehand which can lead to instability during dynamic runtimes
    - Presenters proposed a future direction towards a dynamic heuristic strategy where heuristics are dynamically recalculated during runtime